

Arquitetura de uma aplicação Web e o formato JSON

Objetivo da Aula

Compreender as características do protocolo HTTP e dos aplicativos web, tanto no modelo tradicional como na forma de serviços, além de conhecer a sintaxe JSON para definição de objetos.

Apresentação

O desenvolvimento de aplicativos para web evoluiu muito no decorrer do tempo, e compreender essa evolução nos permite identificar a arquitetura mais adequada a cada contexto de utilização. Mais que isso, conhecer o ambiente de rede e os seus protocolos trará uma melhor compreensão da comunicação com o cliente e do formato de dados adequado em cada situação.

Após descrever os principais protocolos de rede disponíveis, iremos tratar dos aplicativos web, a começar pelo modelo tradicional, em que a interface com o usuário, no formato HTML, é gerada pelo servidor e a entrada de dados é sempre a partir de formulários, podendo adotar estratégia CGI ou Server Pages.

Em seguida, será descrito o modelo baseado em serviços, com web services do tipo SOAP, voltado para o B2B, e do tipo RESTful, amplamente utilizado no B2C e que é baseado nas características do protocolo HTTP.

Finalmente, o formato JSON, utilizado pelos web services RESTful, para a transferência de dados, será detalhado, descrevendo os tipos disponíveis e a implementação de estruturas hierárquicas.

1. Principais Protocolos da Web

Você já deve ter utilizado um navegador, como o Chrome, para visitar os mais diversos sites, saltando para outros endereços nos links da página, assim como acessou seus e-mails

ou baixou alguns arquivos. Por trás de tudo isso, há diferentes programas servidores, todos executando sobre o TCP/IP.

Na verdade, **TCP** e **IP** são dois protocolos que trabalham em conjunto, em que o IP é baseado em um número que identifica o dispositivo, ficando logo acima da camada de enlace, e o TCP efetua o controle de transmissão dos pacotes fornecidos pela camada de aplicação para o IP. Entre os diversos protocolos das aplicações, temos o **HTTP**, para fornecer páginas, o **FTP**, utilizado para transferir arquivos, além do **SMTP** e o **POP**, para enviar e receber e-mails.



Para Refletir

Como é possível saber o tipo de serviço que será utilizado?

Além do endereço IP, é necessária uma porta para efetuar a comunicação, de forma a diferenciar o serviço que será utilizado, como a porta 80 para o HTTP ou a porta 21 para o FTP.

Quadro 1: Exemplos de protocolos e portas associadas

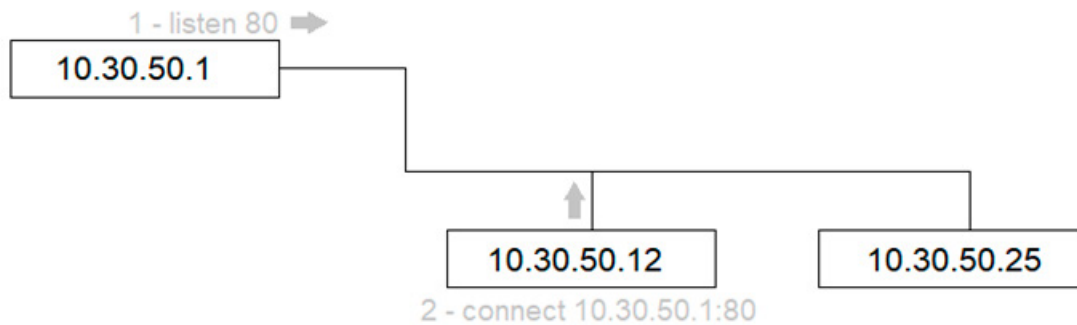
Protocolo	Descrição	Porta
FTP	<i>File Transfer Protocol</i>	21
HTTP	<i>Hypertext Transfer Protocol</i>	80
SMTP	<i>Simple Mail Transfer Protocol</i>	465
POP3	<i>Post Office Protocol</i>	995

Fonte: Elaboração própria.

Ao utilizar portas diferentes, você pode fornecer diferentes serviços em um mesmo endereço, e, embora um servidor como o Apache forneça páginas através da porta 80, outros servidores de páginas podem ser definidos no endereço, desde que utilizem outras portas, como o Tomcat na porta 8080 ou o NodeJS na porta 3000.

O conjunto de IP e porta é denominado **socket**. O servidor deve escutar a porta (listen), no primeiro momento, e o cliente pode solicitar uma conexão, em seguida, indicando o endereço e a porta. Sendo detectada a solicitação pelo servidor, ele deve instanciar um socket para lidar com aquele cliente específico, em geral com o uso de **thread**, e voltar a escutar a rede.

Figura 1: Cliente e servidor baseados em socket



Fonte: Elaboração própria.

Outro serviço importante é o Domain Name System (**DNS**), que atribui nomes de domínios aos endereços IP, pois o cérebro humano tem maior facilidade para guardar texto no lugar de números.

EXEMPLO

Quando você digitar o endereço **https://www.google.com** no seu navegador, ele irá perguntar ao servidor DNS qual o IP correspondente e, a seguir, irá acessar o site através do IP fornecido.

Uma Universal Resource Locator (**URL**) identifica um recurso na internet, de forma exclusiva, como uma página ou um CGI, e sempre é formada pelo domínio acrescido do caminho até o recurso. Já uma Universal Resource Identifier (**URI**) permite a especificação de um recurso completo ou de um fragmento, como a marcação de um local específico da página, muito comum na rolagem até o topo e o final da página quando o conteúdo é extenso.

Figura 2: Exemplo de URL e URI



Fonte: Elaboração própria.

O protocolo HTTP possui diferentes métodos de acesso, sendo **GET** e **POST** os mais comuns, utilizados diretamente nos formulários HTML. Para o método GET, você terá o envio dos

valores do formulário como parâmetros acrescidos após o sinal de interrogação, na própria URL, enquanto no POST os dados são enviados como um pacote de texto em background.

Uma chamada HTTP, no modo GET, com os parâmetros nome e sobrenome, poderia ser **http://localhost/app1/view?nome=Denis&sobrenome=Cople**. No caso de utilizar o modo POST, a chamada seria **http://localhost/app1/view**, e teríamos um pacote com o conteúdo **nome=Denis&sobrenome=Cople**.

Embora o método POST impeça a exibição da informação na URL, o dado não está protegido, pois os pacotes podem ser capturados por ferramentas como **Wireshark**, apresentando o conteúdo em texto claro. Para proteger o dado com criptografia, você deve utilizar o **HTTPS**, que trata da junção do HTTP com um protocolo criptográfico, localizado entre a aplicação e a camada de transporte, como Secure Socket Layer (**SSL**) ou Transport Layer Security (**TLS**).



Destaque

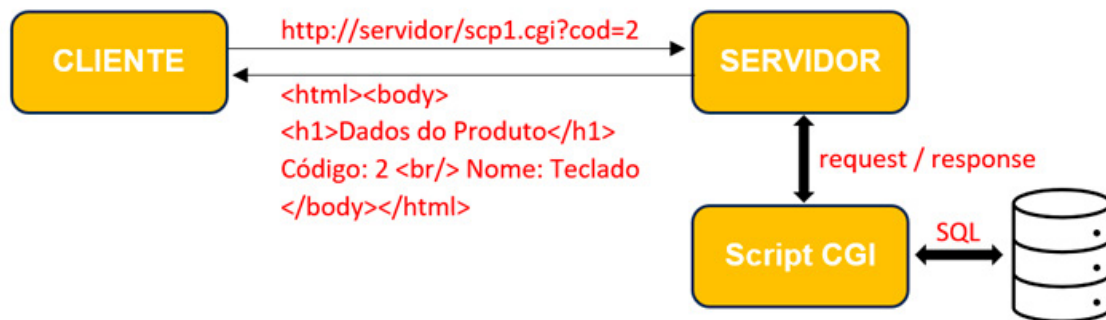
Atualmente, o protocolo SSL está obsoleto, sendo recomendado utilizar sempre o TLS.

2. Aplicativos Web Tradicionais

Na forma tradicional para a criação de aplicativos web, as páginas **HTML** são o meio de entrada e saída de dados. A entrada de dados, no caso, ocorre a partir de formulários, com as informações sendo enviadas para o servidor, onde serão tratadas, mas também ocorrem navegações simples, através dos links.

Ao nível do servidor, você poderá fornecer páginas estáticas ou dinâmicas, em que o conteúdo estático é aquele retornado para o cliente, na sua forma original, enquanto o dinâmico é interpretado pelo servidor ao construir a resposta para o cliente. As primeiras tecnologias comerciais utilizadas com o objetivo de construir conteúdo dinamicamente foram scripts Common Gateway Interface (**CGI**), que são executados pelo servidor em resposta à solicitação do cliente, permitindo o acesso indireto a recursos, como o banco de dados.

Figura 3: Consulta aos dados do produto através de um CGI



Fonte: Elaboração própria.

O protocolo HTTP segue o modelo requisição e resposta, no qual a requisição, ou **request**, pode incluir parâmetros enviados pelo usuário que são recuperados pelos scripts do servidor, e a resposta, ou **response**, permite definir a saída em algum formato de texto interpretável pelo navegador. No exemplo da figura, você pode observar o parâmetro `cod`, com valor 2, na requisição, o qual seria utilizado na definição de um comando SQL de consulta, sendo construída uma página HTML, com os dados do produto solicitado, na resposta ao usuário.



Você Sabia?

A linguagem Perl foi uma das primeiras adotadas na construção de scripts CGI, e o estilo de programação é muito semelhante ao das tecnologias atuais com a mesma finalidade.

Na listagem seguinte você poderá observar um script CGI na sintaxe **Perl**.

```
#!/usr/bin/perl
print "Content-type:text/html\r\n\r\n";
print '<html>';
print '<body>';
print '<h1>Pequeno exemplo de CGI em Perl</h1>';
print '</body>';
print '</html>';
1;
```

Um grande problema no modelo CGI é a construção da resposta através de comandos de impressão para cada linha da saída, exigindo que você encapsule o conteúdo, como HTML, na linguagem utilizada pelo script. Essa abordagem exige que o programador atue, também, como designer, definindo todo o aspecto visual da página.

Com a evolução das tecnologias, surgiu o modelo de páginas do servidor, ou **Server Pages**, tendo como principal característica o encapsulamento de trechos dinâmicos, interpretados ao nível do servidor, no conteúdo da página.

EXEMPLO

Um grande exemplo de tecnologia com essa abordagem é o **ASP**, ou Active Server Pages, da Microsoft, que, durante muito tempo, dominou o mercado.

Um exemplo de código ASP pode ser observado na listagem seguinte.

```
<html>
<body>
<h1>Bem-vindo
<%
response.write(request.querystring("nome"))
%>
</h1>
</body>
</html>
```

Note que o código em Visual Basic fica em trechos demarcados por tags, com o símbolo de percentual, e, como está encapsulado em um documento HTML, é possível utilizar uma ferramenta de criação visual, facilitando o trabalho por parte dos designers. No exemplo da listagem, ocorreria a recuperação do parâmetro **nome**, fornecido via URL, e da sua impressão em meio ao conteúdo HTML que será retornado. Inclusive, se o usuário solicitar a exibição do código-fonte da página, verá apenas conteúdo HTML, sem acesso aos trechos dinâmicos.

As tecnologias continuaram a evoluir, surgiram novas necessidades, em termos de segurança, e o modelo baseado em rotas se tornou um padrão. Com essa abordagem, em vez de acessar diretamente um recurso do servidor, você pode definir caminhos, tratados por

funções do script, que podem incluir trechos dinâmicos para o recebimento de parâmetros de consulta.

Um exemplo de rota com **express**, no **NodeJS**, é apresentado a seguir.

```
const express = require('express')
const app = express()
app.get('/user/:id', (req, res) => {
  res.send(`Id fornecido: ${req.params.id}`)
})
app.listen(3000, () => {
  console.log('Servidor executando na porta 3000')
})
```

Para o exemplo apresentado, se você efetuasse uma consulta ao endereço **http://localhost:3000/user/1001**, no navegador, obteria como resposta o texto **"Id fornecido: 1001"**. Note que o endereço é formado pela URL do aplicativo, acrescida da rota, e que temos um trecho dinâmico no final.

Através do sistema de rotas, os recursos internos do servidor são protegidos, impedindo o acesso direto pelo usuário. Você também poderá exigir que as rotas sejam acessadas apenas por usuários autenticados, facilitando o gerenciamento do controle de acesso no aplicativo.

3. Web Services

Você já ouviu falar de **interoperabilidade**? Esse conceito define a habilidade de os sistemas heterogêneos se comunicarem, segundo protocolos padronizados, e surgiu na área de engenharia, devido à necessidade de conectar dispositivos de diferentes fornecedores em um mesmo equipamento.



Destaque

Em termos de software, com a grande diversidade de tecnologias existentes, a interoperabilidade desempenhou um papel fundamental na criação e conexão de sistemas complexos.

O primeiro exemplo de interoperabilidade na área de software foi a adoção de sistemas gerenciadores de bancos de dados. Diferentes sistemas passaram a compartilhar os dados que são gravados em um mesmo repositório, evitando que as informações fossem replicadas para cada um dos sistemas envolvidos.

No segundo nível de interoperabilidade, temos a execução de processos em máquinas remotas, segundo um modelo de comunicação padronizado. Surgiu o protocolo **RPC**, ou Remote Procedure Call, para dar suporte à disponibilização de processos pelo servidor, com neutralidade de linguagem em termos tanto do cliente quanto do servidor.

Com a popularização da orientação a objetos, os procedimentos que eram executados remotamente passaram a ser fornecidos como métodos de objetos remotos. Surgiram tecnologias de objetos distribuídos, com pools de objetos nos servidores respondendo a múltiplos clientes simultaneamente, além de recursos para controle de acesso e gerenciamento transacional, em que temos como exemplos o Common Object Request Broker Architecture (**CORBA**) e os componentes Java do tipo Enterprise Java Beans (**EJB**).

Ao trabalhar com objetos distribuídos, precisamos de plataformas robustas, normalmente conhecidas como Application Servers, além de um protocolo padronizado e descritores para acesso aos processos. No caso do CORBA, os descritores são criados na sintaxe **CORBA-IDL**, e o protocolo de comunicação é o **IIOP** – ou Internet Inter-ORB Protocol.



Destaque

Com o protocolo e os descritores padronizados, temos neutralidade de linguagem, ou seja, qualquer linguagem que dê suporte a ambos poderá ser utilizada ao nível tanto do servidor quanto do cliente.

Finalmente, chegamos aos serviços distribuídos, definindo serviços que são executados em diferentes servidores, invocados a partir de clientes em qualquer plataforma. Da mesma forma que ocorre para os objetos distribuídos, aqui você terá a garantia de interoperabilidade, devido a um descritor de serviços e a um protocolo padronizado de ampla aceitação no mercado.

No ambiente web, esses serviços são chamados de **Web Services**, podendo ser de dois tipos: **SOAP** e **RESTful**. Os Web Services do tipo SOAP são mais formais, sendo comum você encontrá-los na comunicação entre empresas ou Business to Business (**B2B**), mas o tipo RESTful tem se tornado cada vez mais popular no mercado de desenvolvimento, sendo atualmente o modelo padrão de comunicação Business to Consumer (**B2C**).

Web Services do tipo SOAP trabalham com o protocolo de mesmo nome, que significa Simple Object Access Protocol, baseado na sintaxe **XML**. Utilizam um modelo de requisição e resposta, no estilo RPC, no qual as mensagens utilizam um envelope dividido em cabeçalho e corpo.

Você poderá observar uma **requisição SOAP** na listagem seguinte.

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">
<soap:Body>
<m:Somar xmlns:m="https://servidor/soma">
<m:valorA>10</m:valorA>
<m:valorB>15</m:valorB>
</m:Somar>
</soap:Body>
</soap:Envelope>
```

Essa poderia ser uma chamada para o processo **somar**, disponibilizado por um Web Service, passando os parâmetros **valorA** e **valorB**. Supondo que ocorra a soma dos valores fornecidos, com o retorno através do parâmetro **resultado**, teríamos a resposta que pode ser observada na listagem seguinte.

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">
<soap:Body>
<m:SomarResponse xmlns:m="https://servidor/soma">
<m:resultado>25</m:resultado>
</m:SomarResponse>
```

```
</soap:Body>  
</soap:Envelope>
```

Para que uma plataforma qualquer utilize os serviços, eles serão descritos no formato Web Services Description Language (**WSDL**), que também é baseado na sintaxe XML. A maior parte das plataformas de desenvolvimento apresenta alguma ferramenta para gerar o código do cliente de comunicação a partir de um arquivo WSDL ou de seu endereço na web.

Você poderia descrever o processo de soma anterior através do fragmento de código WSDL apresentado a seguir.

```
<message name="Somar">  
  <part name="valorA" type="xs:int"/>  
  <part name="valorB" type="xs:int"/>  
</message>  
  
<message name="SomarResponse">  
  <part name="resultado" type="xs:int"/>  
</message>  
  
<portType name="SomarPort">  
  <operation name="SomarOperation">  
    <input message="Somar"/>  
    <output message="SomarResponse"/>  
  </operation>  
</portType>
```

Utilizando a sintaxe XML, tanto no protocolo quanto na descrição dos serviços, você terá independência de linguagem, além de transparência para os firewalls, já que a comunicação utiliza apenas elementos de texto.

Com relação aos Web Services do tipo RESTful, ou simplesmente **REST**, eles executam na arquitetura REST – Representational State Transfer –, que utiliza os métodos de chamada do protocolo HTTP para a consulta e manutenção de entidades. Embora seja possível utilizar

XML ou JSON para a representação dos dados de suas entidades, o formato JSON, mais leve e menos formal, foi adotado como padrão pelo mercado.

Para definir um Web Service REST, você irá precisar de uma rota de base, normalmente utilizando o nome do tipo de entidade no plural. Acessando a rota via método **GET**, o Web Service deverá retornar o conjunto de entidades que estão no repositório, utilizando o formato JSON.

Por exemplo, para o endereço fictício **http://servidor/produtos**, você deverá ter uma resposta como a que é apresentada a seguir.

```
[  
  {"id": 1, "nome": "Teclado"},  
  {"id": 2, "nome": "Mouse"},  
  {"id": 3, "nome": "Monitor"}  
]
```

Para o retorno de um produto específico, deverá ser utilizado o endereço de base, seguido do identificador para busca, ainda via método GET. Considerando que a chamada fosse **http://servidor/produtos/2**, teríamos a resposta seguinte.

```
{  
  "id": 2,  
  "nome": "Mouse"  
}
```

A criação de uma entidade deverá utilizar a rota de base, via método POST, com o fornecimento dos dados da entidade no corpo da requisição, adotando o formato JSON. Já a remoção utiliza a rota com o identificador ao final, através do método DELETE.

Por último, temos a modificação da entidade, na rota com o identificador ao final, e o fornecimento dos novos dados no corpo da requisição. Essa operação irá exigir a utilização do método PUT.

Quadro 2: Métodos HTTP, rotas e operações do REST para uma entidade Produto. Fonte: Elaboração própria

Método	Rota	Execução
GET	http://servidor/produtos	Retorna todas as entidades.
GET	http://servidor/produtos/{id}	Retorna uma entidade específica, com o id fornecido na rota.
POST	http://servidor/produtos	Inclui uma entidade, com os dados fornecidos no corpo da requisição.
PUT	http://servidor/produtos/{id}	Modifica uma entidade específica, com o id fornecido na rota e os dados no corpo da requisição.
DELETE	http://servidor/produtos/{id}	Remove uma entidade específica, com o id fornecido na rota.

Fonte: Elaboração própria.

Ao definir Web Services, você irá trabalhar apenas com as informações, sem qualquer preocupação com a interface de usuário. O cliente deverá ser capaz de invocar esses serviços e construir a interface adequada para o usuário.

4. JavaScript Object Notation (JSON)

Originalmente, o formato JSON era apenas uma forma de representar objetos na linguagem **JavaScript**, mas sua leveza e simplicidade fizeram com que fosse adotado para a troca de informações. Esse formato também começou a ser utilizado para armazenagem de informações em alguns bancos de dados NoSQL documentais, como o MongoDB.

No JSON, temos elementos no estilo atributo-valor, em que o nome do atributo e o valor são separados por dois pontos. Para definir um objeto, você deverá utilizar chaves, e os elementos devem ser separados com o uso de vírgula.

```
{
  "id": 1,
  "nome": "Teclado"
}
```

Valores de texto são colocados entre aspas, numéricos sem aspas, e um atributo pode conter um objeto, segundo uma definição recursiva.

```
{  
  "nome": "David", "idade": 45,  
  "endereco": {  
    "logradouro": "Rua Alexandrina, 173",  
    "cidade": "Lagoinha"  
  }  
}
```

Você também pode utilizar valores múltiplos, através da criação de vetores, que são colocados entre colchetes. Devido à natureza recursiva do formato, é possível definir vetores de objetos, como no retorno de entidades pelo REST.

Um exemplo simples de atributo multivalorado pode ser observado a seguir.

```
{  
  "nome": "David", "idade": 45,  
  "dependentes": ["Rachel", "Anna"]  
}
```

Finalmente, podem ser definidos valores booleanos e nulos, como pode ser observado no exemplo seguinte.

```
{  
  "nome": "David",  
  "graduado": false,  
  "faculdade": null  
}
```

Considerações Finais da Aula

Os aplicativos web tiveram grande evolução, ao longo do tempo, buscando aumentar a interoperabilidade e trocando o modelo monolítico inicial por uma arquitetura que faz clara distinção entre back-end e front-end. Os protocolos são os mesmos dos modelos antigos, mas a forma de programar foi modificada.

Vimos que os modelos iniciais eram baseados na apresentação dos dados, mesclando regras de negócio com o código HTML, e que o modelo tradicional considera formulários como meio para entrada de dados, trabalhando apenas com os métodos GET e POST do HTTP.

Porém, com as atuais necessidades dos sistemas, a preocupação passou a ser com o fornecimento de dados e serviços, independentemente da forma como serão apresentados pelo front-end. As tecnologias de Web Services passaram a ser utilizadas, em que os dois tipos principais são o SOAP, voltado para B2B, e o RESTful, normalmente voltado para B2C.

Dada toda essa mudança na forma de projetar os sistemas web, tornou-se necessário um formato padrão para transmissão e recepção de dados, que, no caso, seria XML para o SOAP ou JSON para o RESTful. Ao utilizar esses formatos, garantimos a interoperabilidade, já que são padronizados e estão no modo texto, fazendo com que qualquer ferramenta com suporte à comunicação HTTP seja capaz de definir um front-end que consuma os serviços.

Materiais Complementares

**Vídeo**

Guerreiros da Internet – português

2011, Fábio Nunes dos Santos.

O vídeo trata de uma animação que resume muito bem os conceitos relacionados a redes e internet, com uma apresentação extremamente lúdica.

Link para acesso: <https://www.youtube.com/watch?v=FJDkLAzOiKM> (acesso em 12 set. 2023.)

**Site**

JS JSON

2023, W3Schools (W3C).

O site traz diversas informações e exemplos acerca da sintaxe JSON, com a possibilidade de testar códigos on-line. Deve ser observada a navegação lateral para acesso aos tópicos.

Link para acesso: https://www.w3schools.com/js/js_json_intro.asp (acesso em 12 set. 2023.)

Referências

BARBOSA, C. **Arquitetura TCP/IP I**. Porto Alegre: SAGAH, 2020. Disponível em: <https://abre.ai/gKFh>. Acesso em: 12 set. 2023.

CARDOSO, L. **Frameworks back end**. São Paulo: Platos Soluções Educacionais, 2021. Disponível em: <https://abre.ai/gKFk>. Acesso em: 12 set. 2023.

FERREIRA, A. **Interface de programação de aplicações (API) e web services**. São Paulo: Platos Soluções Educacionais, 2021. nível em: <https://abre.ai/gKFh>. Acesso em: 12 set. 2023.

MAIA, L. **Arquitetura de redes de computadores**. Rio de Janeiro: LTC, 2013. Disponível em: <https://abre.ai/gKFp>. Acesso em: 12 set. 2023.